TELECOM Nancy 2A
Graphes et Recherche Opérationnelle 1

TD MACHINE 3 : COLORATION DE GRAPHES

1 Rappel du problème [1]

Soit $G = (S, \Gamma)$ un graphe non orienté, le problème est d'attribuer une couleur à chaque sommet/nœud $s \in S$ du graphe avec les contraintes suivantes :

— deux sommets s et s' reliés par une arête (donc $\{s, s'\} \in \Gamma$) doivent être coloriés avec deux couleurs différentes $col(s) \neq col(s')$.

— on veut utiliser un minimum de couleurs.

Les couleurs seront représentées par des entiers consécutifs $0, 1, \ldots, nc-1$. Où nc est donc le nombre de couleurs utilisées. Ce problème de coloriage peut se formuler comme un programme linéaire mais qui est souvent difficile à résoudre (pour un graphe suffisament gros). Ainsi plusieurs heuristiques ont été développées. Dans ce TP vous allez coder l'heuristique décrite dans le polycopié (mais qui est rappelée ci-après) puis résoudre aussi le problème comme un PLNE à l'aide de `gurobi`. Pour chaque problème à tester la solution heuristique va permettre d'avoir un peu moins de variables dans le PL.

Quelques définitions

— Le nombre minimal de couleurs défini par ce problème d'optimisation de coloriage s'appelle le *nombre chromatique du graphe*, souvent noté χ.
— On appelle degré d'un sommet x le nombre de ses voisins $deg(x) := card(\{y \in S : \{x, y\} \in \Gamma\})$.
— On note $\Delta G := \max_{x \in S} deg(x)$ le degré maximal du graphe G.

2 La structure de donnée pour les graphes

Vous allez travailler avec une structure de données permettant de représenter un graphe non orienté (et non valué). Cette structure est codée comme une classe python que vous trouverez dans le fichier module `util_tp3.py` disponible sur Arche. Pour une instance `G` de la classe `GrapheElem` :

— `G.nom` est une chaîne de caractères (permettant de donner un éventuel nom au graphe) ;
— `G.donnees` est un dictionnaire (une table de hachage en langage python) dont les clés sont les identificateurs désignant les sommets [2] et les valeurs correspondantes, des objets de classe `Node` détaillée ci-après.

Une instance `node` de classe `Node` décrit les données associées à un sommet (que l'on appellera i dans la suite) ; elle possède les deux champs suivants :

— `node.voisins` qui est un ensemble python [3] dont les éléments (les clés) sont celles des nœuds voisins de i ;
— `node.attr` est un dictionnaire permettant de donner des attributs à chaque sommet ; en particulier dans ce TP si on désire visualiser le graphe, ce dictionnaire doit comporter (pour tous les sommets) la clé `"coord"` associée à une liste de deux nombres donnant les coordonnées $[x, y]$ du nœud ; pour la coloration de chaque sommet vous aurez besoin de rajouter un attribut de couleur (un entier) dont la clé imposée est `"col"` (cette clé est utilisée par la méthode qui permet de dessiner un graphe). Pour savoir si une couleur a déjà été attribuée au nœud `i` on pourra utiliser `if "col" in G[i].attr`. Dans ce cas le numéro de la couleur s'obtient avec `num_col = G[i].attr["col"]`. Pour ajouter un tel attribut de couleur à un nœud (ou pour le modifier) on utilisera :

1. Cf introduction polycopié "un problème posé sur un graphe"
2. Python permet d'utiliser des clés assez variées (entiers, chaînes de caractères, ...).
3. Il s'agit d'un dictionnaire (table de hachage) dont les clés ne sont pas associées à des valeurs.

```
G[i].attr["col"] = 1 # on donne la couleur 1 au n{\oe}ud i
```

Grâce à la surcharge de certaines opérations, manipuler un graphe `G` pourra se faire à l'aide de certains raccourcis d'écriture :

— si `i` est un objet python "hachable", alors l'expression `i in G` retourne vrai si `i` est effectivement (l'identificateur d') un nœud du graphe `G` (faux sinon) et le code :

```
for i in G :
    ......
```

permettra d'itérer sur tous les sommets du graphe `G`

— si `i` est l'identificateur d'un nœud de `G` (et `j` un objet python "hachable") alors `j in G[i]` retourne vrai si `j` est (l'identificateur d') un voisin de `i` et le code :

```
for j in G[i] :
    ......
```

permet d'itérer sur tous les voisins de `i`.

— Pour contre, pour accéder aux attributs il faut utiliser la syntaxe :

```
G[i].attr["coord"]   # pour accéder aux coordonnées du n{\oe}ud i
G[i].attr["col"]     # pour accéder à la couleur du n{\oe}ud i
```

2.1 Quelques méthodes de la classe `GrapheElem`

Entrer [4] les instructions suivantes (sans les commentaires !) :

```
# import de la seule classe GrapheElem
from util_tp3 import  GrapheElem

# construction d'un graphe
G =  GrapheElem(nom="truc")  # une instance vide (enfin juste un nom...)

# on ajoute 4 noeuds (utile surtout pour mettre des coordonnées)
G.ajoute_noeud('s',{"coord" : [-1,0]}); G.ajoute_noeud(1,{"coord" : [0,1]})
G.ajoute_noeud(2,{"coord" : [0,-1]}); G.ajoute_noeud('t',{"coord" : [1,0]})

# on ajoute des arêtes
G.ajoute_arete('s',1); G.ajoute_arete('s',2);
G.ajoute_arete(1,2); G.ajoute_arete(1,'t');
G.ajoute_arete(2,'t')

# on ajoute des couleurs à chaque sommet
for col, i in enumerate(G):
    G[i].attr["col"] = col

print(G) #  doit vous renvoyer ceci :

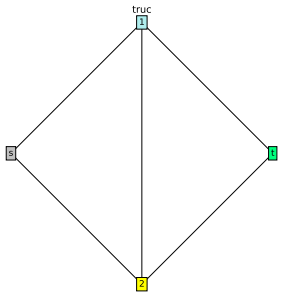
 nom du graphe : truc
 les 4 noeuds :
     id : s, attributs : {'coord': [-1, 0], 'col': 0}
     id : 1, attributs : {'coord': [0, 1], 'col': 1}
     id : 2, attributs : {'coord': [0, -1], 'col': 2}
     id : t, attributs : {'coord': [1, 0], 'col': 3}
 les 5 arêtes :
     {1,t}, {1,2}, {s,1}, {2,t}, {s,2}
```

Pour dessiner un graphe, on utilisera `G.dessine()` (sans coloration, les sommets apparaissent en blanc). Par défaut `G.dessine()` ouvre une nouvelle fenêtre graphique. Si vous précisez `G.dessine(num_fig=1)` la fenêtre 1 sera utilisée (et son contenu initial effacé).

4. Dès l'import effectué, n'oubliez pas la complétion pour éviter de tout taper, par exemple pour la deuxième instruction, vous pouvez taper au clavier la séquence de touches `G = Gr<Tab>` !

```
# dessin du graphe
G.dessine()
```

Vous devriez obtenir :



Ajout puis suppression de nœuds, d'arêtes, ajout d'attributs, etc :

```
len(G) # nb de noeuds du graphe
G.nb_aretes()  # nb d'arêtes du graphe
G.ajoute_noeud(3,{"coord" : [-0.3,-0.2]})
G.ajoute_arete('s',3); G.ajoute_arete(3,'t'); G.ajoute_arete(3,2)
len(G)
G.nb_aretes()
G.dessine()

# rajout/modification d'attributs du noeud s (si un attribut existe déjà sa valeur
# est écrasée par la nouvelle)
G.ajoute_attribut('s',{"people":["pierre","paul","jacques"], "coord" : [-1,-0.2]})

print(G)
G.dessine()

G.enleve_noeud(3)  # enlève aussi toutes les arêtes associées
G.dessine()
G.enleve_arete(1,2)
G.dessine()
```

On voit qu'il est aisé de rajouter toutes sortes de données en chaque nœud.

Pour les tests de coloration de graphes, `util_tp3.py` contient :

- les 4 fonctions `Petersen()`, `Golomb()`, `Reine55()`, `graphe_grille()` qui fabriquent chacune un graphe où les sommets sont munis de coordonnées ce qui permet de les représenter [5] avec la méthode `dessine` ;
- la fonction `read_dimacs` qui permet de lire un graphe au format dimacs (les sommets dans ce format n'ont pas de coordonnées), plusieurs fichiers sont disponibles sur Arche.

Les identificateurs des nœuds de tous ces graphes sont des entiers compris entre 0 et $n-1$ ce qui permet de simplifier un peu la construction des matrices pour la formulation PL.

3 Travail à réaliser

Les diverses fonctions à coder seront écrites dans le fichier `coloration.py` (à récupérer sur Arche) qui contient leurs entêtes et une petite partie du code (entre autres la fonction permetant de construire l'une des matrices de contrainte du PL est déjà écrite).

3.1 Calcul de ΔG

Compléter la fonction `degmax(G)` qui calcule le degré maximal du graphe `G`. Tester votre fonction sur les graphes de *Petersen*, *Golomb* et *Reine55* ($\Delta G_{Petersen} = 3$, $\Delta G_{Golomb} = 6$, $\Delta G_{Reine55} = 16$ et $\Delta G_{Grille} = 7$).

5. Reine55 est un graphe assez dense il est difficile de mettre des coordonnées adéquates pour le visualiser, voir les commentaires de cette fonction pour la signification.

3.2 L'heuristique à programmer

Elle est basée sur un principe glouton et s'énonce[6] :

Pour tout $s \in S$:
(i) on regarde l'ensemble des couleurs déjà attribuées aux voisins de s ;
(ii) on en déduit le plus petit entier naturel qui n'appartient pas à cet ensemble ;
(iii) on attribue cette couleur à s.

On montre que le nombre de couleurs obtenu par cet algorithme est inférieur ou égal à $\Delta G + 1$. Comme on connaît le nombre maximum de couleurs possibles, une façon de réaliser l'étape (ii) est de prévoir un tableau de booléens T de taille $\Delta G + 1$ initialisé à Vrai[7] ce tableau est (partiellement) rempli à l'étape (i), un voisin déjà colorié disposant de la couleur c mettra la case c de T à Faux (un voisin qui n'a pas encore été colorié sera ignoré). Il ne reste plus qu'à trouver l'indice du premier Vrai pour déterminer la couleur de s.

Compléter la fonction `heuristique_coloration(G)` qui implémente cette méthode. La fonction ne doit retourner que le nombre de couleurs utilisées mais elle doit modifier le graphe via l'ajout de l'attribut `"col"` de chaque nœud du graphe (le code contient déjà une initialisation qui enlève les attributs de couleurs au cas où les nœuds ou certains nœuds en seraient déjà munis).

Tests

1. Tester votre fonction sur les quatres graphes avec coordonnées. L'heuristique colore *Petersen* avec 3 couleurs, *Golomb* avec 4 couleurs, *Reine55* avec 8 couleurs et *Grille* avec 5 couleurs. Elle obtient bien les nombres chromatiques pour les deux premiers mais pas pour les deux autres.

   Attention vous pouvez obtenir ces nombres sans que votre code soit juste, pour vérifier :
   — utiliser la fonction `couleurs_distinctes` (présente dans `util_tp3`) qui renvoie vrai si les deux sommets d'une arête sont bien peints avec une couleur différente (et faux sinon)
   — vérifier graphiquement que deux nœuds partageant une arête ne sont pas peints de la même couleur en utilisant la méthode `G.dessine()` sur le graphe après application de l'heuristique (néanmoins c'est peu lisible sur le 3 ème graphe...).

   Ceci peut se faire avec :

```
import util_tp3 as u
import coloration as col
G = u.graphe_grille() # à choisir parmi u.Pertersen(), u.Golomb(), u.Reine55() et u.graphe_grille()
# appel de l'heuristique
m = col.heuristique_coloration(G)
# on vérifie que les couleurs sont bien distinctes
if not u.couleurs_distinctes(G):
    print(" l'heuristique semble bugguée")
else:
    print(f" l'heuristique trouve {m} couleurs")
    G.dessine()
```

2. Tests avec les fichiers dimacs. Ces tests sont effectués par le fichier `tests_heuristique.py` à récupérer sur Arche. Voici ce que vous devez obtenir :

graphe	myciel5	myciel6	david	le450_5a	le450_15d	anna	miles250	inithx
nb cols	6	7	12	14	31	12	9	31
χ	6	7	11	5	15	11	8	31

   Vous devez voir que l'heuristique donne une approximation du nombre chromatique en un temps très court (cf temps de calcul affichés par le script). On voit aussi quelle est exacte pour certains graphes, presque exacte pour d'autres et assez loin pour les deux graphes *le450_5a* et *le450_15d*.

6. Les **couleurs utilisées sont associées à des entiers** $0, 1, \ldots, n_c - 1$ où n_c sera donc le nombre de couleurs utilisées.
7. `T=np.ones(taille,dtype=bool)` créé un tel tableau et `T[:]=True` permet de le réinitialiser par la suite si besoin.

3.3 Résoudre (exactement) par gurobi via une modélisation par PLNE

Soit m un majorant du nombre de couleurs (obtenu par exemple via l'heuristique précédente), et n le nombre de sommets du graphe, on va utiliser les variables binaires suivantes :

1. $x_{i,j}$ valant 1 si on attribue la couleur j au sommet i (0 sinon) ($0 \leq i \leq n-1$, $0 \leq j \leq m-1$) ;
2. y_j valant 1 si la couleur j est utilisée (0 sinon), $0 \leq j \leq m-1$.

La fonction à minimiser est, a prori, le nombre de couleurs, soit $\sum_{i=0}^{m-1} y_i$. Comme pour le problème des clés (cf TP1) la fonction $\sum_{i=0}^{m-1}(i+1)y_i$ qui limite le nombre de solutions optimales est moins performante avec la version actuelle de gurobi... Les contraintes sont les suivantes :

1. deux sommets voisins ne peuvent pas avoir la même couleur, soit :

$$\text{Pour chaque arête } \{i,k\} \in \Gamma, \forall j \in [\![0, m-1]\!] \ : \quad x_{i,j} + x_{k,j} \leq y_j \iff x_{i,j} + x_{k,j} - y_j \leq 0 \quad (1)$$

2. chaque sommet est coloré par une et une seule couleur, soit :

$$\forall i \in [\![0, n-1]\!] \ : \quad \sum_{j=0}^{m-1} x_{i,j} = 1$$

On regroupe les variables x_{ij} et y_j dans un seul vecteur X de $N = (n+1)m$ composantes comme suit :

$$X = (x_{0,0}, x_{1,0}, \cdots, x_{n-1,0} \,|\, x_{0,1}, x_{1,1}, \cdots, x_{n-1,1} \,|\, \cdots \,|\, x_{0,m-1}, \cdots, x_{n-1,m-1} \,|\, y_0, \cdots, y_{m-1})^\top$$

On remarque que $x_{i,j} = X_{i+nj}$ et que $y_j = X_{nm+j}$. Le deuxième groupe de contraintes s'écrit alors $BX = b$ où B est une matrice creuse de taille $n \times N$ et b un vecteur de taille n ne contenant que des 1. Cette matrice, identique à celle intervenant dans les clés usb, est déjà codée dans le fichier `coloration.py`. On va donc se concentrer sur les inégalités (1) qui s'écrivent $AX \leq 0$ (ici 0 désigne un vecteur de 0 de taille le nombre d'inéquations).

Écriture de la fonction qui construit la matrice A

Tout d'abord, répondez ci-dessous aux questions suivantes. Si on appelle na le nombre d'arêtes alors :

Le nombre total d'inéquations est égal à : $nb_{ineq} =$

La matrice A est donc de taille (nb lignes, nb colonnes) :

Combien d'éléments non nuls comporte-telle : $nb_{elts} =$

Écrire $x_{i,j} + x_{k,j} - y_j \leq 0$ en remplaçant les variables par leur équivalent dans le vecteur X :

Compléter la fonction `mat_A` dans `coloration.py` mais lisez bien les recommandations suivantes :

> Cette matrice creuse sera construite en format "i,j,val" (appelé format coo dans scipy), c'est à dire en construisant 3 tableaux I, J, Val chacun de taille nb_{elts} avec $I[\ell]$, $J[\ell]$ et $Val[\ell]$ respectivement l'indice de ligne, l'indice de colonne et la valeur d'un coefficient non nul de A ($A_{I[\ell],J[\ell]} = Val[\ell]$). Pour tous les graphes proposés les identificateurs des n sommets sont les entiers $0, \ldots, n-1$, il n'y a donc pas d'identificateur à traduire en terme d'entier (sinon il faudrait installer un attribut `num` pour chaque nœud). La matrice est construite en parcourant les arêtes ce qui peut se faire de la façon suivante :

```
for i in G:
  for k in G[i]:
    if i < k: pour ne pas parcourir l'arête {i, k} deux fois...
      # coefs de A liés à l'arête {i, k} : peut-être une boucle en j sur les couleurs ?
```

Une partie du code est déjà écrite (il suffit de la décommenter).

Tester votre code en utilisant le graphe de *Petersen* puis en choisissant $m = 3$ (valeur du nombre de couleurs obtenue par l'heuristique). La fonction `mat_A_Petersen` (qui se trouve dans `util_tp3` retourne cette matrice en format plein (numarray), si `A` est la matrice (creuse en format coo) obtenue avec votre fonction alors l'expression :

```
np.all( A.toarray() == mat_A_Petersen() )
```

doit retourner Vrai.

La fonction qui définit et résout le PL avec gurobi

Compléter la fonction `PL_coloration` dans `coloration.py` qui définit et résout le PL de coloration pour gurobi. Cette fonction contient une partie du code (à décommenter) en particulier à la fin pour retourner le nombre chromatique du graphe et installer les couleurs dans le graphe. Pour cela il faut utiliser le même identificateur `PL_col` pour le modèle gurobi.

Tests

— Tester d'abord votre fonction avec les 4 graphes avec coordonnées. Pour l'argument m prendre le nombre de couleurs obtenu via l'heuristique. Sur les deux premiers graphes vous pouvez voir que l'heuristique a trouvé une solution optimale mais pas sur les deux autres, on découvre alors que le nombre chromatique de *Reine55* est égal à 5 et celui de *Grille* est de 4.

— Tests avec les fichers dimacs. Hélas parmi ces graphes seul *myciel5* donne un PL qui "passe" sans licence gurobi [8]. Si vous avez obtenu une licence gurobi les problèmes suivants se résolvent très rapidement :
  — *david* : l'heuristique donnait 12 et on obtient bien $\chi_{(david)} = 11$;
  — *anna* : l'heuristique donnait 12 et on obtient bien $\chi_{(anna)} = 11$;
  — *miles250* : l'heuristique donne 9 et on obtient bien $\chi_{(miles250)} = 8$.

  Par contre les autres graphes (plus gros néanmoins) lui posent des problèmes. A priori celui qui vient en ordre de taille est *myciel6* mais j'ai arrêté le calcul sur ma machine au bout de 2h... On a affaire ici à un problème dont l'approche PLNE n'est pas actuellement pleinement satisfaisante (en attendant peut-être d'autres progrès sur ces solveurs), d'où l'importance des approches heuristiques. Ici on pourrait tester d'autres heuristiques et/ou modifier l'heuristique utilisée mais ce TP est déjà assez long.

  Il est intéressant de suivre les informations sorties par gurobi (par exemple sur le problème *myciel*5). Vous trouverez à cette url :

  `https://docs.gurobi.com/projects/optimizer/en/current/concepts/logging/mip.html`

  les informations pour les décrypter !

8. Nécessaire dès que le PL comporte plus de 2000 variables ou contraintes (d'après `https://support.gurobi.com/hc/en-us/articles/15801588452241-Gurobi-OptiMods-error-Given-data-exceeds-Gurobi-trial-license-limits`).